ChatGPT

Executive Summary

Serverless development is held back by poor local tooling. Tools like AWS SAM CLI and LocalStack partially fill the gap, but developers still face slow feedback loops, missing service emulation, and high resource overhead【9†L335-L343】【7†L121-L128】. We propose an integrated **AWS Lambda Local Testing Platform** – essentially a "VS Code for serverless" – that delivers a **fast, high-fidelity local AWS environment**. Our platform will let backend engineers develop, debug, and test Lambda functions and event-driven workflows entirely on their laptop, with instant hot-reload, rich UI tooling, and broad AWS service support.

Key features include: a single desktop app (built with Tauri or Electron) with a web-style UI; a core engine (in Go or Rust) that spawns language-specific Lambda runtimes via lightweight adapters; built-in mocks of AWS services (SNS, SQS, EventBridge, DynamoDB, S3, API Gateway, Step Functions, IAM, CloudWatch, etc.); and deep integration with IDE debuggers. The product will prioritize performance (no heavy Docker startup delays), accurate service emulation (unlike SAM's limited mocks), and a smooth UX.

We outline personas, use cases, success metrics, and a full set of technical requirements. We compare design options (Tauri vs Electron; Go vs Node; storage engines) in tables, and describe the runtime adapter architecture in detail. Mermaid diagrams illustrate the system architecture and a phased roadmap. Non-functional needs (cross-platform, performance, security) and a testing/CI strategy are covered. Finally, we present a phased implementation plan with milestones, risks, and staffing.

In short: this PRD charts the journey from concept to beta launch of an all-in-one local serverless development IDE, addressing pain points in existing tooling【7†L121-L128】【9†L335-L343】. It leverages proven patterns (we might even embed Moto for some mocks【45†L316-L324】) and modern tech (Tauri/Go or Electron/Node) to give developers the speed and convenience they need.

Product Vision

Enable **backend developers to develop, debug, and test AWS Lambda–based applications locally** as easily as writing code in a text editor. Specifically, the product will:

- **Simulate the AWS cloud locally.** It will host mock implementations of key AWS services (SNS, SQS, EventBridge, DynamoDB, S3, API Gateway, Step Functions, IAM, CloudWatch, etc.) so that Lambda functions can be invoked with realistic events and service interactions. Unlike AWS SAM (which only emulates Lambda function execution in isolation【9†L335-L343】) or LocalStack (which can be slow and heavy【7†L121-L128】), our platform will aim for fast startup and high fidelity.

- **Streamline the edit-build-test loop.** Code changes in Lambdas should reflect instantly (hot reload) without full Docker rebuilds. Invoking functions (via HTTP endpoints, queued messages, or manual events) should feel instantaneous.

- **Provide a unified GUI/CLI experience.** Through a desktop app (Tauri or Electron) or CLI, developers can visualize their serverless architecture, kick off functions, debug code with breakpoints, inspect logs/metrics, and manage mocks – all in one interface.

- **Support real AWS workflows.** We will support multiple runtimes (Node.js, Python, Go, Java/.NET, etc.), and let users import their IaC definitions or connect to AWS credentials to pull down configuration. (At minimum, we'll allow mapping "existing AWS Lambda" into the local env.) The goal is for the local environment to mirror a real AWS account, so integration bugs disappear before deployment.

- **Scale with user needs.** The product starts as a single-developer local tool, but we envision pro tiers for team collaboration (shared projects, cloud sync), similar to LocalStack's move to a freemium model【45†L316-L324】.

This platform is **desktop-native (Windows/Mac/Linux)** and open to all teams (not requiring AWS creds for basic use). By giving teams a "local AWS cloud" that is lightweight and fast, we aim to recover time lost on slow redeploys and AWS configuration. We measure success by developer productivity (minutes saved per debug cycle), adoption rate, and user feedback.

Target Users and Personas

Our primary users are **software and DevOps engineers** building serverless applications on AWS. Likely personas include:

- **Senior Backend Engineer ("Amy").** Teams like yours; she writes production APIs and background services in Python or Node. She frequently needs to debug a Lambda with SQS or DynamoDB triggers. Amy currently uses AWS SAM or LocalStack and finds them slow or incomplete. She wants faster feedback and an IDE-like debugging experience.

- **Full-Stack Cloud Developer ("Raj").** Uses AWS Lambda + API Gateway to serve a React app. Wants to quickly test endpoints and see logs, without constant `sam deploy`. He values hot-reload and a visual UI to manage functions.

- **DevOps Engineer ("Priya").** Writes Terraform/CDK for serverless infra. She wants to run integration tests for Lambdas and event rules locally, to catch issues before merging. She prefers CLI tools and scripting, but would benefit from a dashboard to inspect event flows.

- **Engineering Team Lead ("Carlos").** Oversees a microservices team. He is interested in reducing cloud spending and risk by enabling the team to test offline. Also cares about onboarding: new hires should get the environment up quickly. Carlos might approve a pro license purchase if it saves developer time.

These users share frustrations: **slow iteration** (deploying to AWS takes minutes), **disconnect between code and AWS** (SAM bypasses services【9†L335-L343】, LocalStack containers slow down), and **lack of tooling** (no easy way to visualize Lambdas or step through code locally).

By addressing these pain points, our product will attract any AWS serverless developer (especially those using Python, Node.js, or Go) who wants a "local cloud" dev environment. Surveys on forums

(StackOverflow, r/aws) confirm that fast local loops and realistic mocks are in high demand【7†L121-L128】【45†L316-L324】.

Core Use-Cases

We will support the following key workflows:

1. **Local Lambda Testing (Function-as-a-Service).** Write a Lambda handler (in Node, Python, etc.) and invoke it locally with a test event. View the result immediately. (This is like `sam local invoke`, but with faster startup and ability to mock services.)

2. **API Gateway Emulation.** Define HTTP endpoints mapping to Lambdas. Start a local HTTP server so hitting `http://localhost:3000/...` triggers the correct function. (Similar to SAM's `start-api` or `serverless-offline`, but fully integrated.)

3. **Pub/Sub Simulation (SNS, SQS, EventBridge).** Publish messages to local SNS topics or push messages into SQS queues; the system dispatches these to subscribed Lambdas. This simulates event-driven workflows: e.g. **upload file** → SNS notification → Lambda → **update DB**. No need for a real AWS account.

4. **Data Store Mocks (DynamoDB, S3, etc.).** Provide local databases and buckets. For example, a DynamoDB table is backed by a local on-disk store. Lambdas can perform CRUD on it as if hitting AWS, and changes persist between runs. S3 operations (GET/PUT) are done against local folders. The local data is visible (developers can open a local file to see contents).

5. **Step Functions & State Machine Debugging.** (Advanced) Support AWS Step Functions by allowing state machine definitions to run locally. This could initially be a stub that visualizes steps and checkpoints Lambdas at each state.

6. **Function Debugging & Breakpoints.** Launch Lambdas under a debugger (e.g. Node inspector, Python debugpy). Developers set breakpoints in their IDE and step through code with real function contexts. Integration with VS Code debugger (like AWS Toolkit's remote debugging) would let them pause on line and inspect variables.

7. **Hot Reload / Live Coding.** When code or configuration changes, the platform auto-reloads the affected Lambdas or routes, with minimal delay. This is akin to SST's "Live Lambda" feature【20†L131-L140】, but fully local – no redeploy needed.

8. **Logs, Metrics, and Tracing.** Aggregate logs from each Lambda execution into a UI console, similar to CloudWatch Logs. Also capture execution metrics (duration, memory) and possibly distributed tracing. This lets users quickly see output and performance.

9. **Event Recording & Replay.** Allow users to record an incoming event (HTTP request, SNS message, etc.) and replay it on demand. This supports regression testing of functions with known inputs. The recorded events are listed in the UI and can be re-run with one click or via CLI.

10. **Infrastructure Sync (Cloud Sync).** In a later phase, optionally connect to an AWS account to pull resource definitions (Lambdas, triggers) down to local, or push local changes out. E.g. "Import from CloudFormation" or "Deploy current project".

11. **Collaboration (Team Edition).** In a team setting, share events, mock configurations, or even environment snapshots. For example, one dev can export a set of sample messages that another can import. (This is a pro feature for later phases.)

Each of these use cases builds on core requirements. For now, MVP will focus on local invocation (use cases 1–4) and smooth UI/CLI integration. Then we'll iterate to add debugging and collaboration.

Success Metrics

We will measure success in three categories:

- **Developer Productivity:** Track how much time users save in testing/deploy loops. For example, compare "code change → testable" time with cloud vs our tool. (Aim: reduce cycle time from minutes to seconds.) Survey metrics: NPS score, or "% of devs who prefer local vs cloud iteration".

- **Adoption and Usage:** Monitor downloads/installs and active user count. Target milestones might be "1000+ active projects in 6 months". Usage metrics (optional telemetry) could include number of Lambda invocations per week or number of event flows created.

- **Feature Usage & Feedback:** Which features see heavy use (e.g. API Gateway emulation vs SNS vs SQS)? These inform product priorities. Collect user feedback on stability, fidelity, performance.

- **System Performance:** Internally, measure startup time (target <1s for environment spin-up), function invocation latency, memory footprint. E.g. track "time to first Lambda invocation after start" to ensure it remains low (current SAM local can take many seconds on first run【7†L121-L128】).

- **Reliability:** Track uptime (no crashes) and number of bug/issue reports. For each service mock (SNS, SQS, etc.), define "API compatibility tests passed" – e.g. responses match AWS's behaviors 95% of the time.

We will define specific OKRs for these metrics once we have a prototype. For example:

Metric	KPI Target
Cold-start to ready	<1 second
Lambda invocation time overhead (vs instant)	<100ms per call
Local multi-service workflow correctness	>90% AWS parity
Developer cycle time reduction	5× speedup vs cloud deploy (95th percentile)
NPS (developer satisfaction)	> +40 (by v1.0)

Requirements

Functional Requirements

- **Project/Workspace Management.**
- Developers can create or open a "Lambda project", defined by a folder with function code.
- The system reads a config (e.g. `project.toml` or JSON) specifying Lambdas (name, runtime, handler, code path), triggers (HTTP route, SNS topic, SQS queue, etc.), environment variables, IAM roles, and AWS region defaults.
- CLI: `lambda-local init` to bootstrap a new project.

- **Multi-Runtime Support.**

- Must support **multiple Lambda runtimes** out of the box: at minimum Node.js (v18+), Python (3.8+), Go, Java, and .NET Core.
- Each runtime adapter can start multiple worker processes. E.g. Node adapters may use a pool of `node` processes. Python adapter might use `gunicorn` or similar.
- Adding new runtimes (e.g. Ruby) should be possible via a plugin-like architecture.

- **Service Emulation.**

- **SNS Topics & Subscriptions:** Create topics, publish messages, auto-deliver to subscribed Lambdas. Support raw and JSON formats.
- **SQS Queues:** Create local queues with at-least-once semantics. Producers can send messages, and subscribed Lambdas auto-poll or event-driven trigger similar to AWS Lambda event source mapping. Support dead-letter queues.
- **EventBridge (CloudWatch Events):** Create custom event buses. Put events with detail types. Route to Lambdas on pattern matching (like AWS EventBridge).
- **DynamoDB Tables:** In-process emulation of tables. Basic operations (PutItem/GetItem/Query/Scan). Support Streams to trigger Lambdas on data changes.
- **S3 Buckets:** Local directories act as buckets. `PutObject` writes a file; `GetObject` reads it. Support event triggers (e.g. "on object created" events that invoke Lambdas).
- **API Gateway (HTTP):** Map local HTTP paths/methods to Lambdas. Support REST (integration type "AWS_PROXY"). The local webserver should emulate path parameters, headers, etc.
- **Step Functions (State Machines):** Optional advanced feature. Initially could provide a debugger that steps through states and invokes local Lambdas in sequence.
- **IAM (Authorization):** Simplify by allowing all local code to run with full privileges. (For pro version, possibly simulate IAM policies, but MVP will not enforce them.)

- **CloudWatch Logs:** Aggregate stdout/stderr from Lambdas into a searchable log view. (No actual AWS CloudWatch API needed, just local storage and UI.)

- **Runtime Adapters and Invocation.**

- The core engine receives an "invoke" event (triggered by HTTP, SNS, etc.). It selects the target Lambda adapter (based on runtime).

- The engine communicates with the adapter via an **IPC protocol**. We can use **gRPC** or **JSON-over-stdin/stdout pipes**, or even a lightweight local HTTP (e.g. on `localhost:port`). Each has pros/cons:
    - **gRPC**: Strongly-typed, performant; requires code generation for each runtime binding.
    - **HTTP/REST**: Simple, language-neutral; uses loopback. E.g. the adapter could spin up a mini webserver.
    - **Custom JSON over stdio**: E.g. SAM CLI uses a protocol on stdin/stdout for each container. This is efficient but binds tightly to processes.
- Each adapter should implement the **Lambda Runtime API** semantics. For example, it should call the user handler function with an event JSON, then capture the JSON response or error.
- Adapters must mimic AWS Lambda behavior: e.g. environmental variables (AWS_REGION, AWS_LAMBDA_FUNCTION_NAME), the context object (with memory limit, timeout, AWSRequestId), and return values.

- Adapters should be long-lived: start a process pool and reuse it across invocations (for hot-code reload support).

- **Hot Reload.**

- Detect code changes and automatically restart or refresh the corresponding adapter so new code is used. Should not require restarting the entire engine.

- In UI or CLI, indicate when reloads happen.

- **Debugging.**

- Support attaching a debugger to a running Lambda. For example, for Node, launch with `--inspect`, for Python use `debugpy`, etc.
- Integrate with IDEs: e.g. provide command to generate a `.vscode/launch.json` that points at the local adapter port. (AWS Toolkit does similar in VSCode【13†L429-L437】.)

- Developers set breakpoints in their code; when the adapter hits it, execution stops and they can inspect variables.

- **CLI Interface.**

- Commands (TBD naming):

    - `lls init`: scaffolds a new project (similar to `sam init`).
    - `lls start`: boots the local environment (starts all services, webserver, event loops).
    - `lls stop`: stops the local environment.
    - `lls invoke <function>`: manually invoke a function with a JSON payload.
    - `lls logs <function>`: tail or show recent logs for a function (like `sam logs`).
    - `lls sync`: (future) sync from AWS account.
    - `lls doctor`: check environment health (Docker, etc.).

- **UI/UX (Desktop App).**

- A modern GUI (Electron or Tauri app) with a **Project Explorer** showing functions and resources.
- Panels/Views: Event Tester, Logs console, Metrics, Configuration.
- Drag-and-drop triggers: e.g. UI to create an SNS topic and subscribe a function to it.

- In-app editor: optional simple code view or editor tabs (or just link to external editors).
- Visual debugger: show call stack, variables when paused.

- Diagrams: a flow chart view of event flows (visualization of AWS Step Functions or EventBridge rules).

- **Persistence/State.**

- All configuration (project file), recorded events, and logs should be stored persistently (e.g. in a local SQLite database or JSON). This allows restart-resume and event replay.
- User data (e.g. saved test events, secrets) should be encrypted at rest if containing sensitive info (for security, though local).

Non-Functional Requirements

- **Performance:** Instantaneous developer feedback is critical. Startup time (the time from launching the tool to first readiness) should be <1 second. Function invocation latency (beyond running the actual code) should add only minimal overhead (e.g. <50ms). We will benchmark against existing tools: SAM CLI cold start can be several seconds【7†L121-L128】; our goal is orders-of-magnitude faster.

- **Resource Efficiency:** Avoid heavy dependencies like Docker for every function. The core should be lightweight (written in Go or Rust) and spawn native processes. Memory footprint should be modest (target under 200MB for the core and one runtime).

- **Cross-Platform:** Support Linux, macOS, and Windows (64-bit and Apple Silicon ARM where applicable). We'll use cross-platform frameworks (Go/Rust for core, Tauri/Electron for UI).

- **Security:**

- Run all code in user's permission context (no elevated rights).
- Sandbox adapters as much as possible. For example, if using Docker for Java/Python runtimes, lock down volumes. (Initial MVP can trust the user, but mention as future risk mitigation.)

- Sensitive info (AWS credentials, secrets) must never be sent out. In "offline" mode, no network calls to AWS are made by default. In "sync" mode, use secure AWS SDK calls.

- **Reliability:** The platform should handle errors gracefully (one function error shouldn't crash the whole tool). Provide diagnostics (e.g. `lls doctor`).

- **Extensibility & Maintainability:** The architecture should allow adding new AWS service mocks or languages without rewriting the whole app. Use plugin-like modules or a clear interface for adapters.

- **User Experience:** The UI must be responsive, intuitive, and not feel like a crippled emulation. Developers should not have to wrestle with obscure configs. Good defaults (e.g. most services auto-start) and meaningful errors are essential.

- **Legal/OSS:** If reusing code (e.g. Moto, LocalStack libraries), ensure licenses are compatible (MIT/BSD). The core can be open-source to attract community contributions. Some service implementations (like Cognito, which is complex) may be out of scope.

Supported Runtimes

AWS Lambda currently supports dozens of runtimes, but we'll start with the **most common**:

- **Node.js (v18)**: Popular for APIs. Adapter: spawn `node` processes with `--require aws-lambda-ric` (or use AWS's Runtime Interface Client).
- **Python (3.9/3.10)**: For FastAPI and data tasks. Adapter: use `python` with `aws_lambda_powertools` or AWS's runtime stub. We may embed Moto for AWS SDK calls in Python (since many devs use Python with boto3【45†L316-L324】).
- **Go (1.21)**: Growing use for high-performance Lambdas. Adapter: compile `main.go` to a binary and exec it (as AWS does).
- **Java (OpenJDK 11/17)**: Many enterprises use Java Lambdas. Adapter: run `java -jar function.jar` using AWS Lambda Java runtime.
- **.NET (C# .NET 7)**: Similar adapter with `dotnet`.
- **Others (future)**: Ruby, Rust, etc. We'll design the adapter interface so new languages can plug in (e.g. a generic "binary" adapter).

For each supported runtime, we require:

- The correct AWS Lambda Invocation model (event JSON in, output JSON out) and environment (e.g. `/var/task` volume).
- Support for Lambda layers (initially, simple: mount a `layers/` folder per function).
- Handling cold starts: the first invocation might have initialization code (create adapters accordingly).

Service Mocks and Emulations

We plan to mock/emulate the following AWS services locally. Each mock's behavior should closely match AWS APIs:

- **API Gateway:** Local HTTP server with routes configured from `lls init` or project config. E.g. `GET /users` invokes `UserListFunction`. Support path parameters and query strings. (Under the hood, this is mostly HTTP routing, not too hard.)

- **SNS (Simple Notification Service):**

- Create Topic: API call or UI action creates an in-memory topic.
- Publish Message: `Publish` to topic triggers sending the payload (JSON message) to all subscribers (SQS or Lambda).
- Subscription: Support subscribing a Lambda (or an SQS queue) to the topic. When subscribed to a Lambda, messages result in immediate function invokes. (We can cheat by having SNS mock just forward to adapters directly.)

- Resource: We maintain a list of topics and subscriptions.

- **SQS (Simple Queue Service):**

- Create Queue: API to make a named queue. We back it by an in-memory queue (or persistent list).
- SendMessage: adds message to queue.

- Receive: A subscriber Lambda can poll (like `aws events listen`). For simplicity, we can use long-polling or webhook – e.g. the adapter periodically polls and invokes function.
- Support visibility timeout, deleting after processing, and dead-letter queue.

- Because local dev likely not high throughput, an approximate SQS is acceptable.

- **EventBridge:**

- Create custom event bus.
- PutEvents: send events with `Source`, `DetailType`, and JSON `Detail`.
- Rule matching: Based on event pattern filters (basic JSON matching), route events to Lambdas or to other targets.

- This allows scheduling events or connecting Lambda invocations via intermediary event logic.

- **DynamoDB:**

- Create tables with key schemas. Use an embedded key-value store (e.g. SQLite or HashMap) to store items.
- Support basic operations (PutItem, GetItem, Query, Scan).
- DynamoDB Streams: if enabled on a table, then on any write we generate a stream record. If a Lambda is subscribed, invoke it with the record.

- Must handle typical Dynamo behaviors (eventual consistency vs strong, etc.). At least strong consistency for local simplicity.

- **S3:**

- Create buckets (local directories).
- PutObject/ GetObject: read/write files.
- ListObjects, delete, etc.

- S3 Events: on object creation (via Put), generate an S3 event (with bucket/key info) and invoke any subscribed Lambda.

- **Step Functions:** (Later phase)

- Emulate a state machine definition locally. This is complex; as a start, we can support simple linear workflows. For example, parse a JSON step function, invoke each Lambda in order. We may mark as “preview feature”.

- **IAM (Identity and Access):**

- For MVP, we **don’t enforce** IAM policies (assume full access). In UI, we can let user assign an IAM role name to each function, but not simulate it.

- Alternatively, for pro-tier we might allow testing of authorizer behavior for API Gateway (but not a priority).

- **CloudWatch Logs:**

- Collect all console output (stdout/stderr) from each Lambda invocation.
- Present it in log streams per function. Provide filtering (by time, keyword).
- **CloudWatch Metrics and X-Ray:** (Stretch)
- Possibly gather invocation durations and counts. Could integrate OpenTelemetry to simulate tracing, but very advanced.

The key is broad coverage of common services. We'll initially **exclude** complex ones like Cognito, Kinesis, ElastiCache, etc., since LocalStack itself requires Pro for some of these【45†L316-L324】. We focus on compute and data flows.

Implementation Note: We may leverage existing emulators where possible. For example, **Moto** is a well-known Python library that mocks many AWS services【40†L74-L79】. We could embed Moto's logic (e.g. run a Moto-based process for S3/Dynamo). However, Moto is Python-specific; for the core we're likely in Go or Rust. We could either:

- Ship a Python/Moto subprocess for certain services, communicating over HTTP (Moto runs a local Flask server with AWS endpoints).
- Or port just the needed parts (too costly). For now, we can cite that **Moto is often used for AWS mocks and LocalStack itself uses Moto under the hood for services like S3 and DynamoDB【40†L74-L79】**. We may use Moto in our implementation as a "service pack" (if we build in Python for some).

Adapter Architecture and IPC Protocol

The heart of the system is the **Runtime Adapter** model:

- The **Core Engine** (Go/Rust) acts as a message router. It listens for events (from HTTP, CLI, or timers) and dispatches them to the appropriate Lambda function adapter.
- For each **Lambda function**, we create an adapter that runs the function code. Adapters could be:
- **Process-based:** Start a separate OS process (e.g. `node`, `python`, etc.) that listens for invocation requests via a protocol.
- **Thread-based (for compiled languages):** For Go/Java, maybe start a Java VM or Go plugin in-process. (Go is compiled, but AWS Lambda Go invocations usually run compiled binaries, so likely still separate process).
- **Container-based:** We prefer to avoid Docker containers for speed, but optionally allow running code in a Docker (for languages without easy embedding).
- **IPC Protocol:** We need a way to send the event JSON from core to the adapter and get a response. Several choices:
- **HTTP/REST:** The adapter could expose a small HTTP endpoint (bound to localhost port) that accepts JSON POST and returns JSON. Pros: language-agnostic, easy to implement. Cons: overhead of HTTP stack, port management, potential security hole (should bind only to loopback).

- **gRPC:** Define a proto service. Pros: efficient, many language implementations. Cons: requires proto definitions and code generation for each runtime adapter. It adds dependencies (though Tauri or Go would handle gRPC well, Node and Python too).
- **Named pipes / Sockets:** A simple binary protocol on a pipe or Unix socket. This is fast but requires more custom coding and error-handling.
- **STDIO JSON:** Inspired by AWS SAM, we could run a process and send it JSON via stdin, get response on stdout (each invocation as separate run, or persistent process reading streams). This avoids networking but can be trickier to manage concurrency.

Proposal: Use **HTTP** for simplicity. For each runtime adapter, we launch a sub-process that binds to a random loopback port, and register that with core. Core then `POST`es the event and awaits response. Each adapter process listens indefinitely and handles sequential (or parallel, if multi-threaded) requests.

This approach is similar to how LocalStack+SAM work: SAM's `sam local invoke` spins up a container and calls Lambda handler. Our adapters would use minimal containers or none. The HTTP server in each adapter could be built using simple frameworks (Express for Node, Flask for Python, net/http for Go). Example: a `lambda_adapter.py` script that runs `uvicorn` or similar to serve requests.

- **Startup and Lifecycle:**
- On `lls start`, the core reads config and **pre-starts adapters** for each function. It also sets up service mocks and any required servers (API gateway HTTP server, WebSockets for event stream, etc.).
- Adapters should log their status. If an adapter crashes, the core can show an error and optionally restart it.

- On code changes, the core can restart the affected adapter (like a hot-reload mechanism).

- **Data Flow:** Example: An HTTP GET request hits `http://localhost:3000/users`. The API Gateway emulator captures it, determines it maps to function `GetUsers`. The core then packages an event (API Gateway proxy format) and `POST`s it to the `GetUsers` adapter via HTTP. The adapter runs the handler, returns JSON { statusCode, headers, body }. The core then forwards this as the HTTP response to the original request.

- **Inter-Adapter Communication:** Rarely needed; if one Lambda invokes another, it would normally go through a service (SNS/SQS/EventBridge). So the core's router will handle any such chaining via the mock services. We will *not* allow Lambda A to directly call Lambda B (developers can simulate via SNS/SQS).

```
flowchart LR
    subgraph UI [User Interface (Electron/Tauri)]
        UX[React/HTML UI] -->|Uses| Core[Core Engine (Go/Rust)]
    end
    subgraph Core[Core Engine]
        Core --> API[API Gateway Mock]
        Core --> SNS[SNS Mock]
        Core --> SQS[SQS Mock]
        Core --> DB[DynamoDB Mock]
        Core --> S3[S3 Mock]
        Core --> EF[EventBridge Mock]
        Core --> Logs[Log Store]
```

```
        Core --> Auth[IAM Stub]
        Core -->|invoke| PythonAdapter[Python Adapter]
        Core -->|invoke| NodeAdapter[Node.js Adapter]
        Core -->|invoke| GoAdapter[Go Adapter]
        Core -->|invoke| JavaAdapter[Java Adapter]
    end
    PythonAdapter --> Logs
    NodeAdapter --> Logs
    GoAdapter --> Logs
    JavaAdapter --> Logs
```

Diagram: System components and their relationships. The UI interacts with the Core Engine, which in turn manages service mocks and invokes language-specific adapters. Each adapter runs the user's Lambda code and writes logs back to the Core.

Data Models and Storage

We will store project state in a local **embedded database (SQLite or similar)**. Key entities:

- **ProjectConfig:** Metadata (project name, AWS region default).
- **Function:** (function_id, name, runtime, handler_path, code_directory, environment_vars, memory_mb, timeout_sec, IAM_role).
- **Trigger:** (trigger_id, type [HTTP/SNS/SQS/EventBridge/S3], config JSON, target_function_id). For HTTP, config might include `path` and `method`.
- **ServiceResource:** For named resources like S3 bucket names, DynamoDB tables, etc. This could store schemas (e.g. Dynamo table key schema).
- **RecordedEvent:** (event_id, type, payload, timestamp, result, status). Used for event replay.
- **LogEntry:** (log_id, function_id, timestamp, message, stream). Could be kept in DB or on disk.

SQLite is a strong default choice: it's fast, ACID, supports JSON columns, and needs no separate server. (SQLite is "the most widely deployed embedded database in the world"【38†L48-L57】.) For a pure-Go implementation, BoltDB or Badger could be alternatives, but SQLite's ubiquity (and possibility to view with SQLite browser) is appealing. We'll implement an abstraction so swapping it is easy.

Storage choices:

Option	Type	Notes
SQLite	Relational	ACID, SQL queries, file-based. Widely used. Can store JSON fields. Heavyweight C library but proven.
BoltDB / BadgerDB	Key-Value	(Go-only) Go-native, simple K-V. Good performance, but non-relational.
LevelDB / RocksDB	Key-Value	High-performance C++ libraries. Good for big data, but more complex integration.
JSON/YAML files	Flat-file	Simplest (just write JSON), but no indexing or multi-field query. Harder to evolve schema.
Turso (Rust)	Relational (SQLite-compatible)	New Rust DB, but too new for trust.

We will start with SQLite (using libsqlite3), which works on all OSes and can handle our scale. SQL schema (using `serde_json` where needed) will be versioned for migrations.

For logs, we may append to files and index them, or write to a log DB table. Initial MVP: write logs per function to daily rolling text files (like `/tmp/lambda_logs`) and show them in UI by tailing.

Logging and Tracing

All function stdout/stderr is captured by the adapter and forwarded to the core. We will:

- **Timestamp and tag** each log line with function name and invocation ID.
- In the UI, show live tail (for running functions) and history (in log view). Provide filtering (search, function filter).

For deeper observability:

- Optionally embed an OpenTelemetry agent in each adapter to produce trace spans (each Lambda invocation is one span). This would allow a flame-graph view (stretch goal).
- At minimum, we'll record invocation metrics (duration, memory). The core will expose a metrics endpoint (Prometheus format) so users can scrape with `lls metrics`.

Event Replay

Users can click on a past event (HTTP/SNS/SQS) and **replay** it: the core will re-invoke the same Lambda(s) with the original event payload. We'll store recorded events in the DB (`RecordedEvent` entity). Replay is a simple fetch+invoke. The UI might show a list of events (sorted by time) with ability to edit and resend them.

User Flows (UI/CLI)

We will sketch out a few UI flows:

- **First-run (Initialization):** User runs `lls init` in an empty folder. CLI prompts for project name, optionally scans AWS SAM/Serverless templates to import, or creates a blank template. It generates `lambda-project.yml` with placeholders.

- **Add Lambda:** In UI or by editing config, user adds a function: names it, selects runtime (Node/Python/Go/Java), points to code directory. UI may offer a template (e.g. "new Node.js function").

- **Configure Trigger:** User sets up triggers. E.g. "Create API Gateway endpoint": UI shows a form (path `/hello`, method GET) and attaches it to `HelloFunction`. Under the hood, a Trigger entry is saved.

- **Run Locally:** User hits "Start Local Env" (or `lls start`). The UI shows a status (started, with port number). CLI prints "Local API at http://localhost:3000".

- **Invoke & Debug:** User opens browser or uses curl to hit the endpoint. The UI highlights the function invocation (maybe blinking icon). The Logs panel shows output. If a breakpoint is set, the function pauses and VS Code opens at that line.

- **View Data:** If function wrote to Dynamo or S3, UI shows the stored items or files (similar to S3 Explorer in AWS toolkit).

- **Write Code:** Developer edits code in VS Code. Hot reload triggers rebuild; UI console shows "reloading function X... done". Then user invokes again without manual restart.

- **Recording Events:** In UI's event tester, user can input a JSON payload and press "Publish to SNS" or "Send to SQS". These appear in a history list.

Below is a simplified architecture diagram to illustrate components (see the code block above).

CLI Commands (Examples)

We envision a CLI (`lls` for "Local Lambda Suite" or similar):

- `lls init [--template TEMPLATE]` – Create new project. (Options: select runtime language scaffold.)
- `lls start` – Launch local server and start all services.
- `lls stop` – Terminate all services and adapters.
- `lls invoke <FunctionName> --event event.json` – Invoke function manually.
- `lls logs <FunctionName> [--follow]` – Show recent logs (tail if `--follow`).
- `lls list functions` – List deployed functions and triggers.
- `lls config` – Show environment (e.g. region, project path).
- `lls sync` – (beta) Sync definitions from AWS.
- `lls doctor` – Run health checks (e.g. "Docker not needed / connectivity ok").

Each command should have useful help text and examples.

Packaging: Tauri vs Electron

We must choose a desktop framework for the UI layer. The main contenders:

Aspect	Electron (Node.js)	Tauri (Rust backend)
Binary Size	Large (~>100 MB) due to bundling Chromium + Node【35†L258-L261】.	Tiny (~<10 MB) since it uses native WebView and Rust. (Case: one app dropped from 120MB → 8MB【35†L258-L261】.)
Memory Usage	High (each window uses Chromium).	Low (uses system WebView). Much lighter RAM use【35†L258-L261】.
Development DX	Single tech stack (JavaScript/HTML/CSS). Huge ecosystem and maturity. Hot-reload easy with JS.	Hybrid stack: front-end still JS/HTML, but backend logic in Rust. Requires Rust tooling. Steeper learning curve but very performant【35†L175-L183】.
Performance	Good runtime performance (just JS on V8). But startup and idle footprint is heavy.	Excellent performance at runtime. Rust core is fast and memory-safe. App launches faster.

Aspect	Electron (Node.js)	Tauri (Rust backend)
Security	Node integration can be risky (need context isolation). Many known hardening best-practices.	More secure by default (no Node.js in WebView, limited API surface)【35†L271-L279】. Rust safety helps.
Ecosystem & Plugins	Very mature: thousands of libraries (Node modules) and Electron plugins for auto-update, notifications, etc.	Growing: fewer out-of-box libs, but covers basics. Tauri-specific plugins exist (auto-update, etc.)【35†L241-L249】.
Cross-platform maturity	Battle-tested on Win/Mac/Linux.	Supported but newer; may have edge cases (especially on older platforms).
Upgrade path	If needed, moving to Tauri later would require porting backend to Rust.	Front-end code is mostly transferable; but backend logic needs re-implementation in Rust.

Takeaway: **Tauri** offers substantial advantages in app size, speed, and security【35†L258-L261】【35†L334-L340】. Given this is a developer tool, the smaller footprint and faster launch are big wins. The main trade-off is needing Rust expertise. Since we are focusing on a high-quality product and the user is already experienced with Go, using Rust for the core is feasible. We can also mitigate by writing core logic in Go (compiling to a single binary) and having Tauri call it via JSON RPC or CLI.

Recommendation: Use **Tauri** with a Rust core (or Go core invoked by Rust). This will keep installers small (<50MB on disk) and memory usage minimal. The UI will be built in React/TypeScript (since the user likes React), which Tauri supports.

Table from [35] summarizing conclusion: “Electron shines for ecosystem maturity; Tauri excels in performance, security, minimal resource usage”【35†L334-L340】. Given our performance-centric goals, Tauri is the winner.

Comparison Tables

Frontend Framework

Factor	Electron	Tauri
App Size	~100–150 MB (Chromium bundle)	~5–10 MB (uses OS WebView)
Memory Usage	High (Chromium per window)	Low (native web engine)【35†L258-L261】
Performance	Good, but slower cold start	Excellent (fast startup, small JS bundle)
Security	Needs Node.js isolation	More secure by default【35†L271-L279】
Ecosystem & Tooling	Very mature, lots of plugins	Growing, Rust ecosystem is smaller

Factor	Electron	Tauri
Dev Experience	All-JS stack, easy for JS devs	Requires Rust knowledge, but UI dev remains JS-based
Conclusion	Stable, mature (used by VSCode)	Lightweight, future-proof (best for performance)【35†L334-L340】

Core Language (Backend Engine)

Factor	Node.js (JavaScript)	Go (Golang)	Rust
Startup & Latency	Warm (requires V8 init)	Fast native binary	Fast native binary
Performance (CPU)	Slower for tight loops【43†L143-L152】	High (native, great for CPU-bound)【43†L143-L152】	Very high (similar to C++)
Concurrency	Event-loop + callbacks (good for I/O)【43†L285-L294】	Goroutines (excellent, true parallelism)【43†L285-L294】	Async/threads (excellent)
Memory Usage	Higher (runtime overhead)	Lower	Lower
Ecosystem	Huge (npm modules)	Good (standard lib, some cloud tools)	Growing (fewer libraries, but strong performance libs)
Dev Ergonomics	JS-friendly (familiar)	Static-typed, simple tooling	Steeper (lifetimes), but very safe
Cross-compile	Less straightforward	Excellent (go build)	Good (cargo cross, but toolchain complexity)
Conclusion	Good for quick dev, prototyping, but heavy for production-grade tools	Great balance: fast, concurrent, easy to deploy; used by many CLIs	Top performance and safety, good choice if Rust skills available

We lean toward **Go or Rust** for the core. Both compile to single binaries and are memory-efficient. If team is comfortable with Go, that's an excellent choice; if aiming for best performance/security, Rust (especially if using Tauri) is ideal. The user knows Python and AWS, but not sure on Go vs Rust. We may start with Go for developer productivity, then consider Rust for rewrites of performance-critical parts.

Local Storage Options

DB Option	Type	Language Support	Pros	Cons
SQLite	Relational	All (via libraries)	ACID, SQL queries, embeddable, ubiquitous【38†L48-L57】	Dependency on SQLite library, more overhead than KV
LevelDB	Key-Value	Go, Node, Python	Very fast for simple K-V, small footprint (C++)	No query support, only K-V API
RocksDB	Key-Value	C/C++, via wrappers	High performance (SSD-optimized)	Larger binary (C++), GPL-licensed (or Apache fork)
BoltDB	Key-Value	Go only	Pure Go, simple K-V, ACID via bbolt	Go-specific, no query language
BadgerDB	Key-Value	Go only	Very fast, LSM-based, TTL support	Go-specific, more complex than Bolt
Turso	SQL	Rust	SQLite-compatible, ACID, includes advanced features (CDC)	Very new, less battle-tested
LMDB	Key-Value	C	Memory-efficient, B+tree, very fast reads	Not as popular, works as C lib
In-memory	N/A	Any	Fastest, no persistence (for stateless data)	Data lost on shutdown, not persistent

Recommendation: Use **SQLite** for configuration and record-keeping (stable, cross-platform, easy backup). For high-throughput data needs, BoltDB or RocksDB could be options, but likely not needed. SQLite's popularity (the "most widely deployed embedded DB"【38†L48-L57】) makes it safe.

Architecture Diagram

```
flowchart TB
    subgraph Desktop App
        direction TB
        UI[React UI] --> |User actions| Core[Engine (Rust/Go)]
        UI --> |CLI Commands| Core
    end
    subgraph Engine/Core
        direction LR
        Core --> |routes| APIGW[API Gateway Mock (HTTP Server)]
        Core --> SNS[SNS Mock Service]
        Core --> SQS[SQS Mock Service]
        Core --> |invoke| DynDB[DynamoDB Mock]
        Core --> |invoke| S3Mock[S3 Mock]
        Core --> EventB[EventBridge Mock]
        Core --> StepF[Step Functions Stub]
```

```
    Core --> IAM[ IAM (stub) ]
    Core --> Logs[Log Collector]
end
subgraph Runtime Adapters
    PythonLambda[Python Runtime Adapter]
    NodeLambda[Node.js Adapter]
    GoLambda[Go Adapter]
    JavaLambda[Java Adapter]
    .NETLambda[.NET Adapter]
end
Core --> |HTTP/gRPC| PythonLambda
Core --> |HTTP/gRPC| NodeLambda
Core --> |HTTP/gRPC| GoLambda
Core --> |HTTP/gRPC| JavaLambda
Core --> |HTTP/gRPC| .NETLambda
PythonLambda --> Logs
NodeLambda --> Logs
GoLambda --> Logs
JavaLambda --> Logs
.NETLambda --> Logs
```

Diagram: The Core Engine manages mocks and invokes function adapters. Adapters send logs back to the core. The UI interacts with the core via direct calls or CLI.

Phased Implementation Plan

We propose a **phased rollout** with clear milestones. Each phase builds on the previous:

- **Phase 0 – Research & Design (1–2 weeks):**
- Interview developers, survey existing tools (LocalStack, SAM, SST) to validate pain points 【20†L131-L140】.
- Define MVP scope and architecture.
- Set up a project skeleton (e.g. a GitHub repo, basic CLI).

- Deliverable: Tech design docs, PoC architecture sketch, and initial project repo.

- **Phase 1 – Core MVP (4–6 weeks):**

- **Engine & CLI:** Implement the Core in Go or Rust. Build basic command parser.
- **Function Adapters:** Create first adapters for Node.js and Python (the most common). They should accept an event JSON and run a simple handler (e.g. AWS Hello World).
- **API Gateway:** Build a minimal HTTP server routing to Lambda.
- **Logging:** Capture Lambda output to console.
- **Storage:** Integrate SQLite and save project config.
- **Hot Reload:** Watch code files and restart adapters on change.

- **Deliverable:** `lls init`, `start`, `invoke`, `logs`, and a UI skeleton showing the project tree. Must successfully start a sample Lambda and route an HTTP call.

- **Phase 2 – Core Expansion (4–6 weeks):**

- **More Adapters:** Add Go and Java runtimes.
- **Service Mocks:** Implement SNS, SQS, and DynamoDB mocks (basic versions). Each should trigger Lambdas appropriately.
- **EventBridge:** Add simple event bus routing.
- **UI Integration:** Flesh out the desktop app UI (React) – function list, log view, event tester UI.
- **Debugger Support:** Add ability to attach Node and Python debuggers (expose `--inspect` and `debugpy`).

- **Acceptance Criteria:** Core supports a multi-step workflow: e.g. sending SNS message triggers Lambda which writes to Dynamo. Logs are visible. Debugging stops at breakpoint.

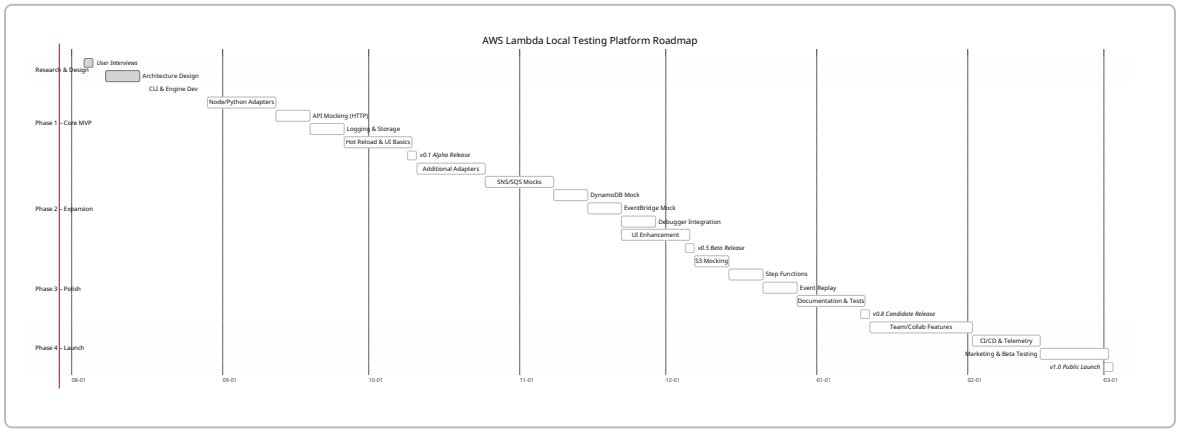
- **Phase 3 – Advanced Features (4–6 weeks):**

- **Additional Mocks:** S3 with bucket events, basic Step Functions flow, CloudWatch logs (UI filters), IAM stub features.
- **Event Replay & Persistence:** Implement event recording and replay UI.
- **UI Polish:** Diagram view of event flows, detailed settings panel.
- **Testing:** Write automated integration tests (simulating function calls).
- **Beta Prep:** Hardening, installers packaging (Tauri/Electron), minor bug fixes.

- **Deliverable:** Beta release (v0.8) with complete core workflow. Collect early feedback.

- **Phase 4 – Beta & Team Features (6 weeks):**

- **Team Edition:** Add project sharing (via cloud or file sync), user accounts (if needed).
- **CI/CD & Telemetry:** Set up CI pipelines (build, tests), integrate optional telemetry.
- **Go-to-market:** Write docs, record launch videos, coordinate marketing (blog posts on AWS blog / Medium about “building our tool”).
- **Launch:** Public v1.0 on Product Hunt / AWS blogs, with support plans.
- **Deliverable:** Production-ready v1.0 with documentation, sample code, and a stable packaged application.



Timeline: Phased development plan with milestones. First alpha by mid-Oct 2026, beta by Dec 2026, v1.0 launch by early Mar 2027.

Deliverables and Effort

We estimate a small dedicated team will be needed (roles are approximate):

- **Backend Engineer (1–2)**: 12–16 weeks total. Building the core engine, service mocks, adapters. Expertise in Go or Rust. (40–60% of timeline work each.)
- **Frontend Engineer (1)**: 8–10 weeks. Building the React/Tauri UI, wiring to core via IPC/HTTP. Creating CLI UX. (Mostly in Phase 2–3.)
- **DevOps/QA (shared)**: Part-time (2–4 weeks spread) for setting up CI/CD, writing automated tests, packaging installers for Windows/macOS/Linux.
- **Product/Project Manager (0.5)**: 4–6 weeks to coordinate, write docs, plan go-to-market.

Total: ~30–40 engineer-weeks. (If one full-time person, ~7–10 months; better with a small team in parallel.)

Risks and Mitigations

- **Feature Creep (AWS Breadth):** AWS has hundreds of services, and they evolve daily【45†L276-L283】. We risk trying to emulate too much (like Cognito, Kinesis). *Mitigation:* Focus on core Lambda-related services initially. Make architecture pluggable so unsupported services simply error out or defer to cloud. Emphasize "production parity where it matters most" – capturing errors early for common cases.
- **Performance Issues:** Spawning many adapters or heavy mocks could slow down local dev. *Mitigation:* Benchmark continuously (e.g. "spin up 10 Lambdas in X ms"), profile hotspots, optimize critical paths (e.g. use in-memory databases where needed, reuse processes).
- **Cross-platform Bugs:** Differences in Windows vs Linux for file paths, ports, etc. *Mitigation:* Test on all target OSes; use Go/Rust which compile on each; use Electron/Tauri which handle OS nuances.
- **Security/Isolation:** Running arbitrary code on user's machine. *Mitigation:* Run everything with user permissions only; do not download or execute untrusted code. Clearly warn users that it runs local code (like any dev tool).
- **Adoption Barrier:** Many devs have established workflows (SAM, LocalStack). We need to show clear value. *Mitigation:* Publish benchmarks and tutorials (e.g. "AWS Lambda in Production: Part 1/2/3" via Medium). Engage early adopters, maybe seed with open-source projects or give free licenses to influencers.
- **Competition:** If AWS or LocalStack improves their offerings (e.g. AWS local emulation). *Mitigation:* Differentiate by focusing on developer experience (UX, IDE integration, speed). And being open-source-friendly (LocalStack's change in licensing【45†L316-L324】 suggests some devs want OSS alternatives).

Go-to-Market and Beta Launch

- **Beta Testing:** We will start a private beta by December 2026. Invite known experts (e.g. participants from Serverless communities, attendees of AWS events). Use feedback to polish the UI and fix critical bugs.

- **Content Marketing:** As we build, publish dev diaries on Medium and GitHub (like this PRD). E.g. “Why we built a Local AWS for Lambdas,” “Deep dive on local SNS emulation,” etc. These attract users and rank on search for related queries.

- **Social Channels:** Announce alpha and beta on r/aws, r/devops, Twitter, Hacker News. Possibly partner with Serverless community sites (ServerlessLand, AWS Heroes).

- **Documentation:** Provide getting-started guides, examples, a cookbook. For example, importing an existing SAM project and running it locally.

- **Pricing:** We can follow a **freemium model** similar to LocalStack【45†L316-L324】:

- *Free Edition:* Core features (single project, community service mocks, local host).
- *Pro Edition (£15/month):* Extra mocks (proprietary services, SSL support, usage analytics).
- *Team Edition (£40/month):* Collaboration (shared projects, cloud sync, priority support).

Exact tiers TBD with user feedback. The initial goal is sign-ups and engagement, not immediate monetization.

Conclusion

Building a robust **AWS Lambda Local Testing Platform** involves careful architecture and phased delivery. By leveraging proven components (Moto, SQLite, React) and focusing on developer pain points (speed, completeness), we can deliver a compelling product. LocalStack and AWS SAM highlight the demand【7†L121-L128】【9†L335-L343】, and emerging tools like SST’s Live Lambda inspire us to bridge the gap【20†L131-L140】.

With this PRD and roadmap, we have a clear path to a production-ready tool: an intuitive IDE-like environment where writing Lambdas feels as fast and safe as coding for on-prem servers.

Sources: We drew on AWS documentation and blogs【9†L335-L343】【13†L469-L477】, LocalStack docs【1†L303-L311】【45†L316-L324】, developer forum discussions【7†L121-L128】【45†L316-L324】【23†L203-L207】, and performance comparisons【43†L143-L152】【38†L48-L57】. These informed our requirements and design choices.
